

归芯科技

GX31X Software Workshop

Confidential

2023/8/21

深圳 · 上海 · 北京 · 天津 · 成都

归芯科技

目录

- Architecture
- Boot
- OSA
- Memory Layout
- Network
- AT
- UART
- GPIO
- Debug
- Camera

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.



归芯科技

Architecture

Confidential

深圳 · 上海 · 北京 · 天津 · 成都

GX316 SDK特点

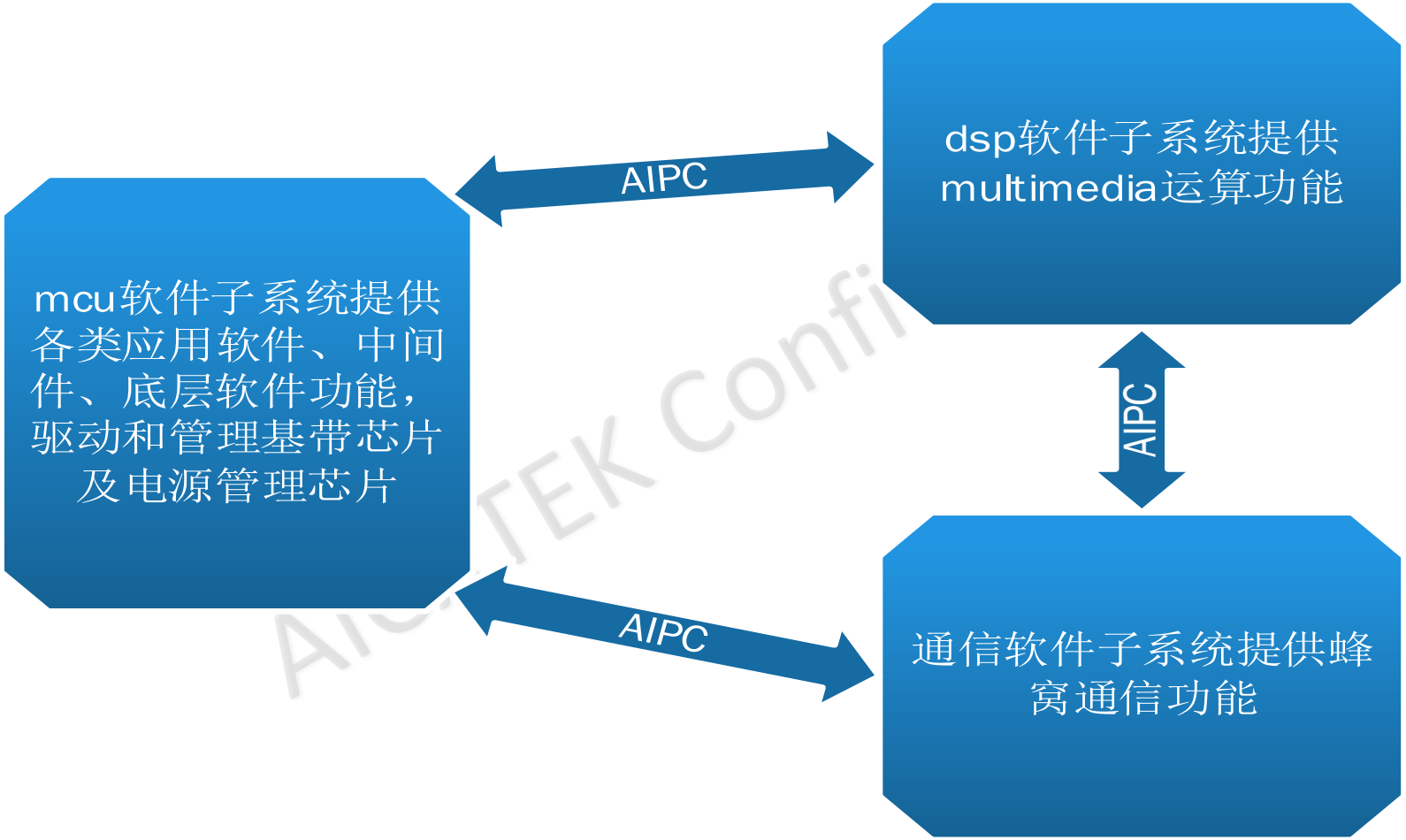
归芯科技

- 免费编译器
- 可视化功能选择配置
- 软硬结合，释放更多mips
- 支持POSIX及CMSIS
- Network、USB、Security等各类复杂功能支持齐全

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

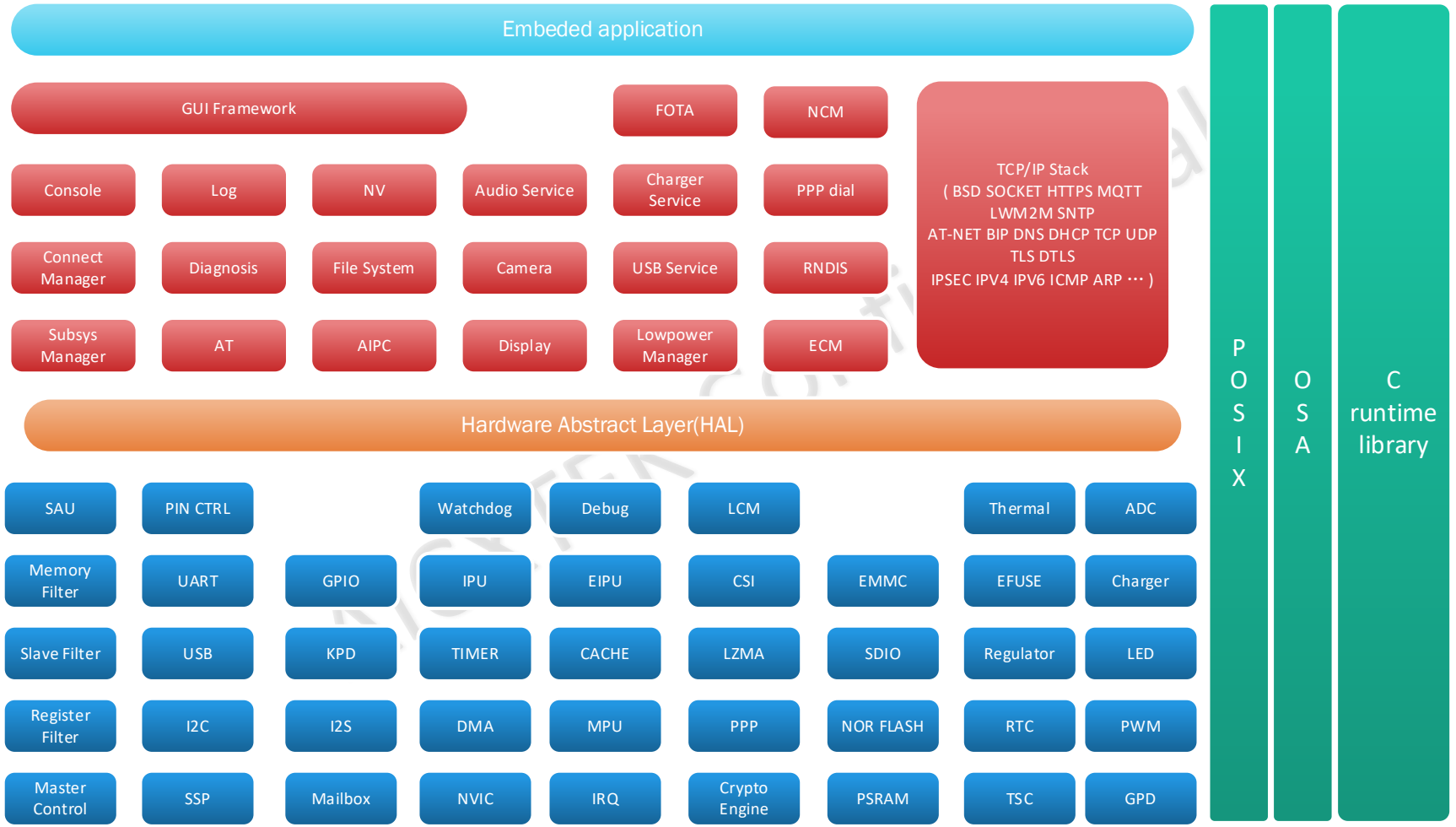
GX316软件框图



Confidential @ 2022 Guixin Technology Co., Ltd.

GX316软件模块层次图

归芯科技



Confidential @ 2022 Guixin Technology Co., Ltd.



归芯科技

Boot

Confidential

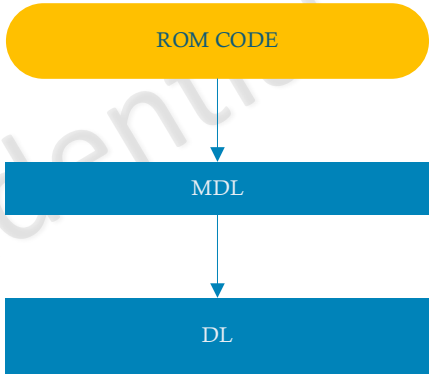
深圳 · 上海 · 北京 · 天津 · 成都

Boot过程概述

归芯科技

download阶段：

- ROM CODE
- Micro download
- Download



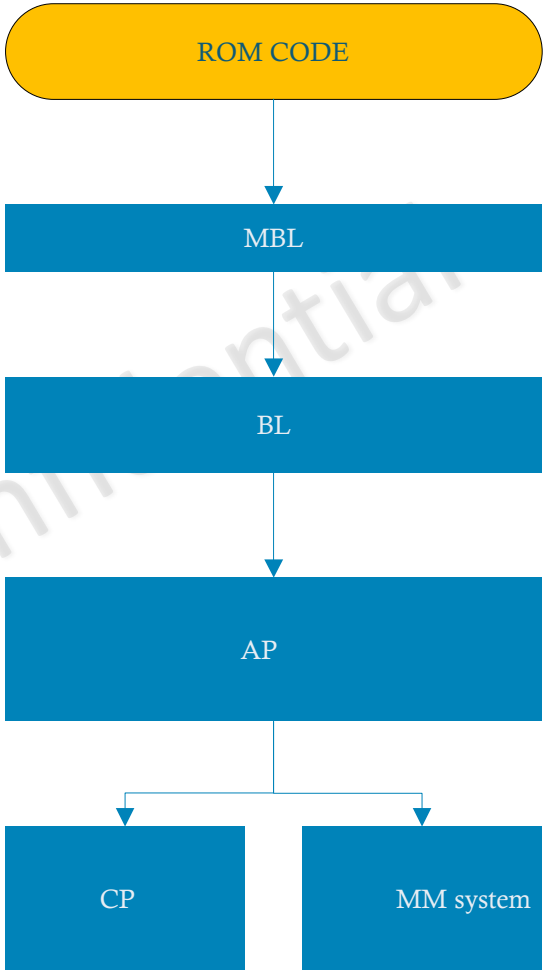
AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

Boot过程概述

boot阶段：

- ROM CODE
- Micro bootloader
- Bootloader
- AP
- CP
- MM system



AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

Boot过程概述

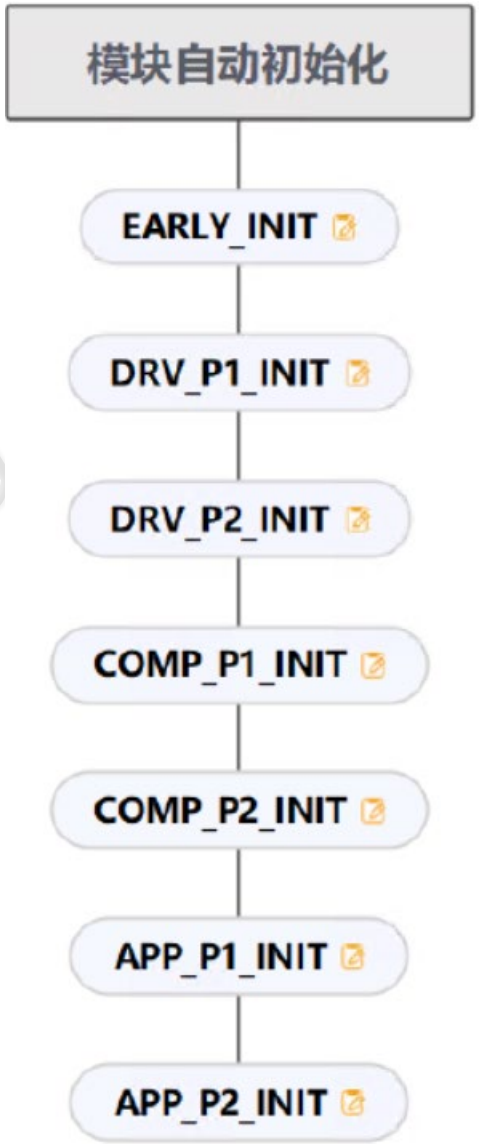
归芯科技

一、AP 大系统 auto init 各阶段：

- early init
- driver init
- component init
- app init

其中 driver、component和 app 又分别分了2个子阶段，
每个阶段的init 顺序如右图。

最基础的，其它模块可能都会依赖的驱动可以放在early init, 其它的一般依赖关系 是：App 依赖 Component 和 Driver，Component 依赖 Driver。
同级别有依赖关系的可以是：被依赖的放P1，依赖的放P2。
注意：同一 级别的init先后顺序不确定。
所有Init 函数是在没有开系统调度阶段执行的，不能使用 Task 上下文安全的 API。



ArcTEK Confid

Confidential @ 2022 Guixin Technology Co., Ltd.

Boot过程概述

二、AP 大系统 later init 各阶段：

- later driver init
- later component init
- later app init

later init 使针对需要使用一些 task 上下文安全 API 的场景增加的，其执行上下文是一个高优先级的init task, 整体的Init 顺序都晚于非later init 所有级别的 init 顺序。later init 里面的各级别的 init 顺序和非 later init 各级别的 init 顺序一致，且每个级别里面也分别有 L1和L2两个阶段，L1早于L2.

ALCATEL Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

Boot过程概述

AP 大系统 Auto init 的使用：

1. 包含头文件。
include <aic_init.h>

2. 导出符号
用对应级别的导出符号宏导出即可（见下图）。

```
AIC_EARLY_INIT_EXPORT(func)
AIC_DRV_P1_INIT_EXPORT(func)
AIC_DRV_P2_INIT_EXPORT(func)
AIC_COMP_P1_INIT_EXPORT(func)
AIC_COMP_P2_INIT_EXPORT(func)
AIC_APP_P1_INIT_EXPORT(func)
AIC_APP_P2_INIT_EXPORT(func)
AIC_MODEM_INIT_EXPORT(func)
AIC_DRV_L1_INIT_EXPORT(func)
AIC_DRV_L2_INIT_EXPORT(func)
AIC_COMP_L1_INIT_EXPORT(func)
AIC_COMP_L2_INIT_EXPORT(func)
AIC_APP_L1_INIT_EXPORT(func)
AIC_APP_L2_INIT_EXPORT(func)
```

3. 加载顺序建议：
一般来说：越基础的驱动模块，越需要放在最前面。
1）如果某个模块/APP不被别的模块初始化时所依赖，并且并不要求尽量早的初始化的话，建议放到later init阶段，用XXX_LX_INIT声明，这个阶段时task上下文，限制更少一些；
2）如果某个模块要求尽量的早执行，就使用xxx_Px_INIT声明，但要保证不要调用可能会导致阻塞的API，否则会报错。

4. 例子

```
int demo_conmponets_init(void)
{

    alog_info("demo componets init.\n");

    return AIC_SUCCESS;
}
AIC_COMP_P1_INIT_EXPORT(alog_later_init);
```

GUIXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

Boot过程概述

Auto init 的扩展：

前面章节的auto init宏是针对所有开机模式下的，另外扩展了一些auto init宏，用于特定开机模式的精细化控制：

```
/* only normal mode init define.*/
AIC_NORMAL_XXX
/* disable normal mode init define.*/
DIS_NORMAL_XXX
/* only basic mode(FOTA) init define.*/
AIC_BASIC_XXX
/* disable basic mode(FOTA) init define.*/
DIS_BASIC_XXX
/* only charge init define.*/
AIC_CHARGE_XXX
/* diable charge init define.*/
DIS_CHARGE_XXX
/* only gxem init define.*/
AIC_GXEM_XXX
/* diable gxem init define.*/
DIS_GXEM_XXX
/* extend image init define, = normal only.*/
AIC_EXTIMG_XXX
```

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.



归芯科技

OSA

Confidential

深圳 · 上海 · 北京 · 天津 · 成都

目录

1 OS及OSA

2 主要组件介绍

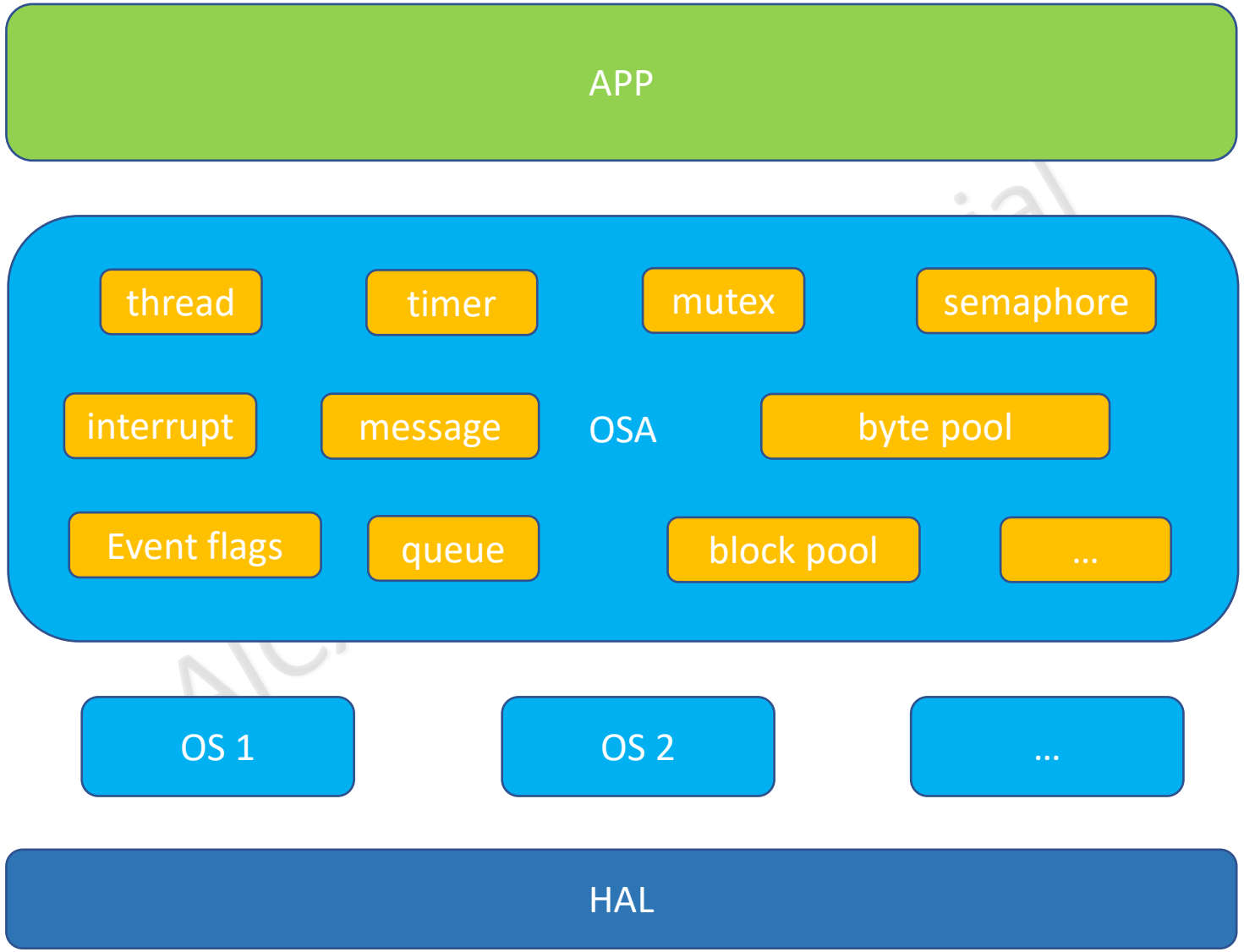
3 API使用示例

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

OS及OSA

归芯科技

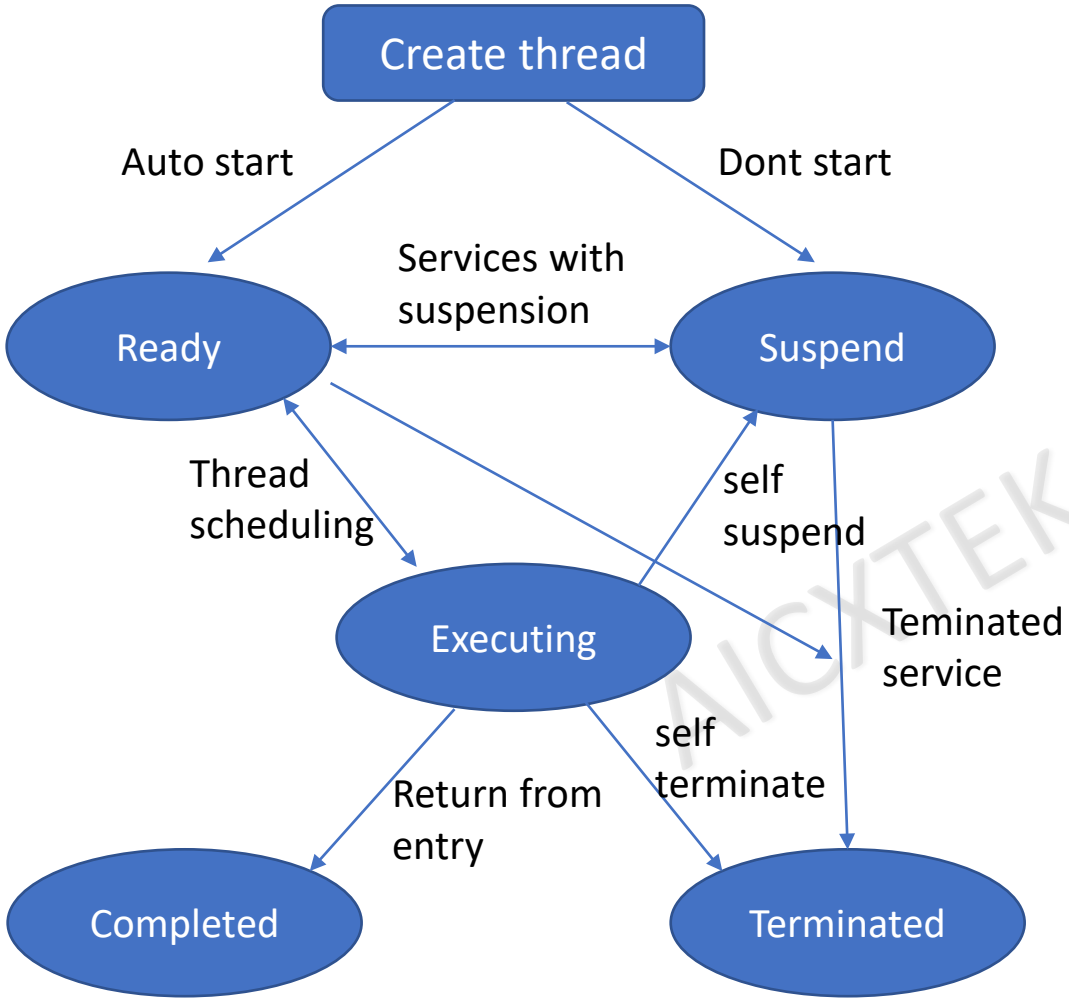


Confidential @ 2022 Guixin Technology Co., Ltd.

主要组件介绍

归芯科技

- Thread




```
aos_thread_id aos_create_thread(
char *thread_name,
aos_entry_func entry_func,
void *input,
u32 stack_size,
u32 queue_num,
char *queue_name,
u32 priority,
u32 autostart)

priority支持256级，规则如下：
0 - 9：    系统核心线程
10 - 19：系统关键组件
20 - 29：用户驱动程序
30 -  ：    用户应用程序
```

```
aos_create_thread
aos_create_thread_simply
aos_delete_thread
aos_change_thread_pri
aos_identify_thread
aos_suspend_thread
aos_resume_thread
aos_terminate_thread
aos_is_thread_created
```

Confidential @ 2022 Guixin Technology Co., Ltd.

主要组件介绍

归芯科技

➢ Timer

提供了单次和周期性timer功能，精度为1ms

```
aos_timer_ptr aos_create_timer(
char *name_ptr,
aos_expiration_func expiration_func,
void *input,
u32 expiration_time,
u32 auto_activate,
boolean b_period)
```

aos_create_timer
aos_delete_timer
aos_activate_timer
aos_deactivate_timer
aos_change_timer
aos_get_remaining_time
aos_is_timer_created
aos_get_timer_info

Confidential @ 2022 Guixin Technology Co., Ltd.

主要组件介绍

归芯科技

➢ Time

提供了tick(ms)、utc time、system time的get/set功能。

```
u32 aos_get_tickcount()
u32 aos_time_to_tick(u32 time)
u32 aos_tick_to_time(u32 tick)
void aos_get_utctime(aos_systime *time)
void aos_set_utctime(aos_systime *time)
void aos_get_systime(aos_systime *time)
void aos_set_systime(aos_systime *time)
```

注：systime是指基于utctime和时区信息计算的Local system time。

Confidential @ 2022 Guixin Technology Co., Ltd.

主要组件介绍

归芯科技

➢ Mutex

实现对临界区的互斥访问

```
aos_mutex_ptr aos_create_mutex(char *name_ptr)

void aos_delete_mutex(aos_mutex_ptr mutex_ptr)

aos_status aos_get_mutex(aos_mutex_ptr mutex_ptr, u32 wait_option)

void aos_put_mutex(aos_mutex_ptr mutex_ptr)
```

Confidential @ 2022 Guixin Technology Co., Ltd.

主要组件介绍

归芯科技

➢ Semaphore

32 bit计数，get/put会进行++/--，semaphore为0时，get的task会被挂起

```
aos_semaphore_ptr aos_create_semaphore(char *name_ptr, u32 initial_count)

void aos_delete_semaphore(aos_semaphore_ptr semaphore_ptr)

aos_status aos_get_semaphore(aos_semaphore_ptr semaphore_ptr, u32 wait_option)

void aos_put_semaphore(aos_semaphore_ptr semaphore_ptr)
```

Confidential @ 2022 Guixin Technology Co., Ltd.

主要组件介绍

归芯科技

➢ Event

提供了32 bit的event flags group，主要用于任务间的事件同步。
每个bit都对应一个事件，可以同时对多个事件进行**OR/OR_CLEAR/ AND/AND_CLEAR**操作



Event flags group

```
aos_event_ptr aos_create_event(char *name_ptr)

void aos_delete_event(aos_event_ptr event_ptr)

aos_status aos_get_event(aos_event_ptr event_ptr, u32 requested_flags,
                         u32 get_option, u32 *actual_flags_ptr, u32 wait_option)

void aos_set_event(aos_event_ptr event_ptr, u32 flags_to_set, u32 set_option)
```

Confidential @ 2022 Guixin Technology Co., Ltd.

主要组件介绍

归芯科技

➢ Queue

Message Queue主要用于任务间通信，queue中可以保留1个或多个消息。
消息的大小可以自定义（1-16 DWORD）




```
aos_queue_ptr aos_create_queue(char *queue_name, u32 item_size, u32 queue_num)

void aos_delete_queue(aos_queue_ptr queue_ptr);

aos_status aos_recv_from_queue(aos_queue_ptr queue_ptr, void *item_ptr, u32 wait_option)

aos_status aos_send_to_queue(aos_queue_ptr queue_ptr, void *item_ptr, u32 wait_option)

aos_status aos_send_to_queue_head(aos_queue_ptr queue_ptr, void *item_ptr, u32 wait_option)
```

Confidential @ 2022 Guixin Technology Co., Ltd.

主要组件介绍

归芯科技

➢ Message

主要用于向指定的Task发消息，基于Queue进行封装，消息的大小为固定的1 DWORD，用于存放指针。

```
u32 aos_send_msg(aos_thread_id dest_id, aos_msg_ptr src_msg_ptr, u32 wait_option)

u32 aos_send_msg_head(aos_thread_id dest_id, aos_msg_ptr src_msg_ptr, u32 wait_option)

aos_msg_ptr aos_recv_msg(aos_thread_id dest_id, u32 wait_option)
```

注：每个task在创建时可指定是否创建queue（可选），然后可以向该task的queue发送msg。

Confidential @ 2022 Guixin Technology Co., Ltd.

主要组件介绍

归芯科技

➢ Memory Management

基于TLSF算法实现了高效低碎片化的memory管理。

主要的API有：
aic_malloc(size)
aic_zalloc(size)
aic_free(mem_ptr)
aic_dma_malloc(size)
aic_dma_free(mem_ptr)

注：dma malloc/free接口主要用于DMA及其他加速器的操作，返回的地址以及实际申请的size都是cache line对齐的。
调用者需要根据需要对该内存进行cache invalid/clean等操作，确保内存一致性。

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

主要组件介绍

归芯科技

- Interrupt

提供了中断使能、禁止，以及中断的注册等服务。

注：中断处理分为上半部和下半部，上半部
运行在中断模式，需要运行的尽可能的快，
下半部运行在thread上下文，可以执行
相对耗时长的处理。

AOS_INTERRUPT_DECLARE
AOS_INTERRUPT_DISABLE
AOS_INTERRUPT_RESTORE
aos_request_irq
aos_request_irq_with_half_bottom
aos_release_irq

```
void aos_request_irq(int irq, int (*handler)(void *p), void *p)

void aos_request_irq_with_half_bottom(int irq, int (*handler)(void *p),
                                      void (*half)(void *p), void *p)

void aos_release_irq(int irq)
```

Confidential @ 2022 Guixin Technology Co., Ltd.

API使用示例

归芯科技

- 部分OSA API的使用，可以参考如下demo文件。
- 具体API原型参见<aic_osa.h>和 <aic_osa_ex.h>



osa_demo.c

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.



归芯科技

PSRAM 和 Flash 的 Customization

Confidential

深圳 · 上海 · 北京 · 天津 · 成都

Customization: psram （一）

归芯科技

AP 的 PSRAM使用概况

如下图，主要分为RO、RW（包含ZI）和Heap三部分，可以根据需要进行调整。

注：size仅供参考，以SDK中的实际配置为准。

```
/*************************************************************
·········ap·psram·mem·layout:·(0x80000000,·0x00180000)
-------------------------------------------------------
·············code·|·(0x80000000,·0x00040000)
-------------------------------------------------------
·············heap·|·(0x80040000,·0x00070000)
-------------------------------------------------------
·············rw···|·(0x800C0000,·0x000D0000)
-------------------------------------------------------
*************************************************************/
```

Confidential @ 2022 Guixin Technology Co., Ltd.

Customization: psram （二）

归芯科技

如何查看各段的使用情况：

编译完后可以从命令行看到当前的使用情况：

Memory region	Used Size	Region Size	%age Used
VECTOR_REGION:	5038 B	8 KB	61.50%
STACK_REGION:	4 KB	4 KB	100.00%
FLASH:	733468 B	784 KB	91.36%
PSRAM_RO:	12879 B	256 KB	4.91%
PSRAM_RW:	673041 B	832 KB	79.00%
PSRAM_HEAP:	448 KB	448 KB	100.00%

这里只需要关注PSRAM_RO 和 PSRAM_RW,

从上图可以看到这两段的大小分别是 256K 和 832K，

PSRAM RO 还有近250K空间，PSRAM RW 不到200K的空间。

Confidential @ 2022 Guixin Technology Co., Ltd.

Customization: psram （三）

归芯科技

各段的大小修改方法：

在目录 aicos/project/genesis/ap/xxx/config下的 sys_mem_cfg.h 中

直接调整各个段的 SIZE 就可以，每段的起始地址会自动计算。

```
49: #define CODE_REGION_BASE ........ SYS_PSRAM_REGION_BASE
50: #define CODE_REGION_SIZE ........ 0x00040000
51: #define LOAD_REGION_BASE ........ (CODE_REGION_BASE)
52: #define LOAD_REGION_SIZE ........ (CODE_REGION_SIZE + HEAP_REGION_SIZE)
53:
54: /*************************************************************
55: ................. Heap memory region
56: *************************************************************/
57: #define HEAP_REGION_BASE ........ (CODE_REGION_BASE + CODE_REGION_SIZE)
58: #define HEAP_REGION_SIZE ........ 0x00070000
59:
60: /*************************************************************
61: ................. rw memory region
62: *************************************************************/
63: #define RW_REGION_BASE .......... (HEAP_REGION_BASE + HEAP_REGION_SIZE)
64: #define RW_REGION_SIZE .......... 0x000D0000
```

Confidential @ 2022 Guixin Technology Co., Ltd.

Customization: psram（四）

归芯科技

MPU的修改方法：

目前MPU配置表会直接用上一章节中配置的宏，MPU无需额外修改。

相关配置文件目录：aicos/project/genesis/ap/xxx/config下的 mpu_map.h

```
    /* PSRAM: AP need dump all ps ram, and need load subsys code. */
    /* code */
    {MPU_RG(CODE_REGION_BASE, CODE_REGION_SIZE),
     RDONLY, MEM_WBWA,   EXEC,       RG_ENABLE},

    /* HEAP + rw + bss . */
    {HEAP_REGION_BASE, AP_REGION_BASE + AP_REGION_SIZE - 1,
     RDWR,   MEM_WBWA,   NO_EXEC,    RG_ENABLE},
```

Confidential @ 2022 Guixin Technology Co., Ltd.

Customization: psram（五）

归芯科技

代码固定在PSRAM的方法：

AP 默认的code 编译是放在 FLASH 上的，如果某些代码必须要在psram上运行（比如flash驱动）或性能要求较高，可以将其指定到psram上执行，

修改方法：aicos/project/genesis/ap/xxx/aic_board.ld。

以下供参考:

```
__psram_text_load_start__ = ALIGN(__idle_load_end__ , 4);
.psram_text : AT(__psram_text_load_start__)
{
    __psram_text_start__ = .;
    snor* (.text .text.*)
    qspi.* (.text .text.*)
    gigadevice.* (.text .text.*)
    winbond.* (.text .text.*)
    aic_mcx.* (.text .text.*)
    string.* (.text .text.*)
    * (.mpu_text)
    __psram_text_end__ = .;
} > PSRAM_RO
__psram_text_load_end__ = __psram_text_load_start__ + SIZEOF(.psram_text);
```

其它没有指定的代码RO 部分会自动放到FLASH，RW会自动放到PSRAM RW。

Confidential @ 2022 Guixin Technology Co., Ltd.

Customization: flash（一）

归芯科技

FLASH分区配置文件介绍

目录：targets/genesis/xxx/fmd_partition_table.json

主要分区如下：

1. ap分区：存放编译的ap.bin

主要是fota升级相关的code，比如基础驱动，OS，http协议等；

2. ap_extend分区：存放编译的ap_extend.bin

主要是fota升级不相关的code，如上层应用等。

在fota升级期间会复用该分区临时存储差分包，以及用于掉电备份。

3. userdata分区：用户数据分区

”C”：内置文件系统分区

“misc_user”：AP侧的NV分区，用于存储配置信息

4. “fota”分区：用于存储fota parameter信息

注：block_num的单位size是4KB，比如312，意为312*4K=1248KB

```
"ap": {
    "block_num": 312,
    "attribute": "RO"
},
"fota_bak": {
    "ap_extend": {
        "block_num": 500,
        "attribute": "RW"
    }
},
"calibration": {
    "block_num": 8,
    "attribute": "RW",
    "discription": "lx_sector_size = 512: total s
    "ftl_table": {
        "nvram_calibration": {
            "sector_num": 32
        },
        "misc_protect": {
            "sector_num": 16
        }
    }
},
"userdata": {
    "block_num": 553,
    "attribute": "RW",
    "discription": "lx_sector_size = 512: total s
    "ftl_table": {
        "C": {
            "sector_num": 4100
        },
        "misc_user": {
            "sector_num": 32
        }
    }
},
"rundata": {
    "block_num": 15,
    "attribute": "RW",
    "discription": "lx_sector_size = 512: total s
    "ftl_table": {
        "misc_run": {
            "sector_num": 32
        },
        "nvram_normal": {
            "sector_num": 64
        }
    }
},
"fota": {
    "block_num": 2,
    "attribute": "RW"
},
"factory": {
```

AICXTEK Confiden

Confidential @ 2022 Guixin Technology Co., Ltd.

Customization: flash（二）

归芯科技

修改分区大小

修改fmd_partition_table.json中中对应分区的”block_num”即可。

由于总的FLASH SIZE 是有限的，某分区增大了，其它就需要减少。

比如：ap分区从312增加到320，ap_extend就需要从500减小到492

```
"ap": {
    "block_num": 312,
    "attribute": "RO"
},
"fota_bak": {
    "ap_extend": {
        "block_num": 500,
        "attribute": "RW"
    }
},
```

Confidential @ 2022 Guixin Technology Co., Ltd.

Customization: flash（三）

归芯科技

增加分区（1）：

可参考下图中 “gnss”分区，增加一个配置项，里面包含”block_num”和”attribute”属性即可。

注意：要从现有分区减小对应的size，确保总的size不变。

```
"resource": {
    "block_num": 13,
    "attribute": "RO"
},
"gnss_bl": {
    "block_num": 4,
    "attribute": "RO"
},
"gnss": {
    "block_num": 75,
    "attribute": "RO"
},
"ap": {
    "block_num": 312,
    "attribute": "RO"
},
```

增加分区（2）：

如果新增分区需要烧写对应的bin文件，还需要修改targets/genesis/xxxxx/pkg/pkg_cfg.json，将该bin打包到pkg中。可以参考下图”gnss”的配置：

```
"resource": {
    "flash_type": "qspi_nor",
    "description": "resource image file"
},
"gnss_bl": {
    "flag": 1,
    "select": 1,
    "bin_path": "ap/bin/gnss_bl.bin",
    "flash_type": "qspi_nor",
    "description": "gnss bl image file"
},
"gnss": {
    "flag": 1,
    "select": 1,
    "bin_path": "ap/bin/gnss_fw.bin",
    "flash_type": "qspi_nor",
    "description": "gnss image file"
},
```

Confidential @ 2022 Guixin Technology Co., Ltd.

Customization: flash（四）

归芯科技

分区调整对校准和FOTA的影响

1. 只要”calibration”分区的配置不变，分区调整对校准就不会有影响。

```
"calibration": {
    "block_num": 8,
    "attribute": "RW",
    "discription": "lx_sector_size = 512: total sector_nu
    "ftl_table": {
        "nvram_calibration": {
            "sector_num": 32
        },
        "misc_protect": {
            "sector_num": 16
        }
    }
},
```

2. 当前的FOTA方案要求新老版本的分区必须要保持一致。

AICYTE Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.



归芯科技

Network

Confidential

深圳 · 上海 · 北京 · 天津 · 成都

归芯科技

目录

1 网络介绍

2 网络组件

3 网络流程图

4 Socket API

5 TLS/DTLS 模块

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

01 网络介绍

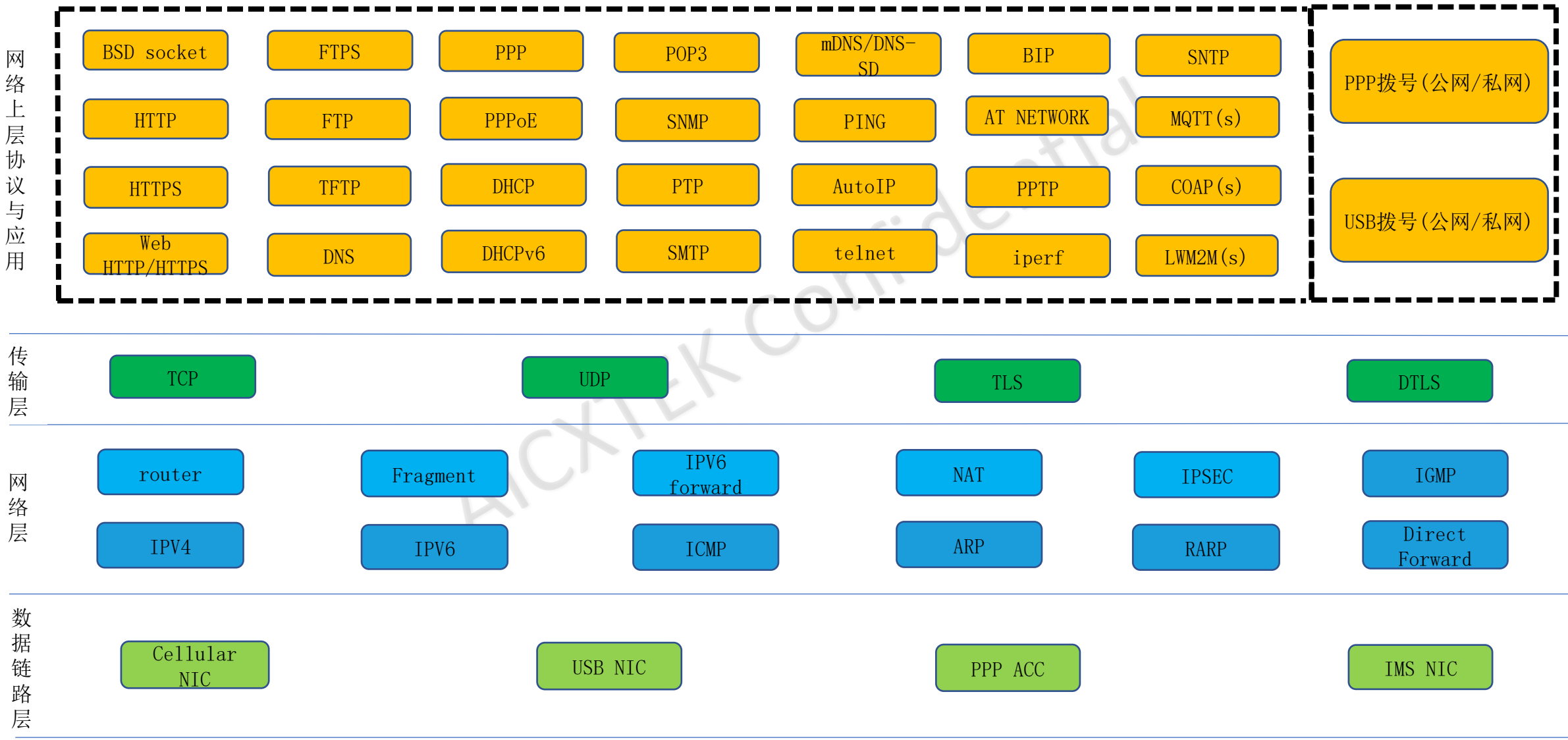
归芯科技

- 本平台网络系统以NETXDUO TCPIP协议栈为核心，具备IPV4 & IPV6双网络堆栈，支持TCP、UDP、ARP、RARP、ICMP、IGMP、NAT、IPSEC等机制，以及完善的SSL/TLS(1.0、1.1、1.2、1.3)/DTLS(1.0、1.2)网络安全协议，同时搭配丰富的上层网络应用协议，可以帮助客户快速完成产品的开发和落地。

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

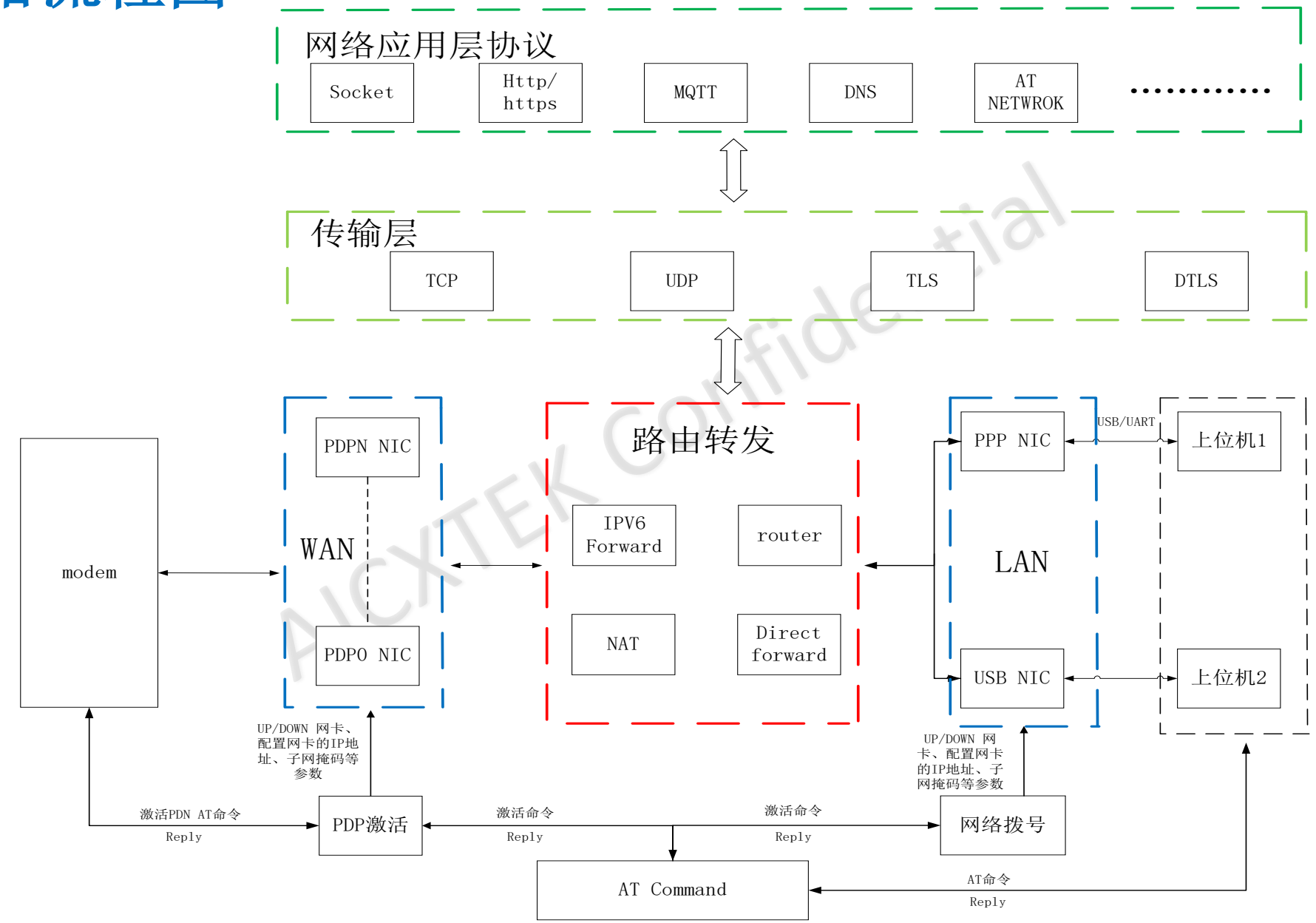
02 网络组件



AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

03 网络流程图



AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

04 Socket API

归芯科技

- 目前网络组件中SOCKET支持的BSD Socket API封装和标准一致，目前支持的主要API如下所示：

Socket API	描述
socket	创造某种类型的套接字，分配一些系统资源，用返回的整数识别，若创建TLS/DTLS socket,需要使用专门的API。
connect	用在客户机这边，给套接字分配一个空闲的端口号。比如说一个TCP套接字，它会试图建立一个新的TCP连接
bind	一般是用在服务器这边，和一个套接字地址结构相连，比如说是一个特定的本地端口号和一个IP地址
listen	用在服务器一边，导致一个绑定的TCP套接字进入监听状态
accept	用在服务器这边。从客户机那接受请求试图创造一个新的TCP连接，并把一个套接字和这个连接相联系起来
send	用来发送数据
sendto	用来发送数据
recv	用来接收数据
recvfrom	用来接收数据
soc_close	关闭连接，系统释放资源，若删除TLS/DTLS socket,需要使用专门的API。
select	用来检测一个或多个套接字的状态。对每一个套接字来说，这个调用可以请求读、写或错误状态方面的信息。

Confidential @ 2022 Guixin Technology Co., Ltd.

05 TLS/DTLS 模块

归芯科技

- NETXDUO中对TLS/DTLS的支持比较全面，其支持的版本和加密套件如下所示：
- 1. TLS1.0
- 2. TLS1.1
- 3. TLS1.2
- 4. TLS1.3
- 5. DTLS1.0
- 6. DTLS1.2

支持的算法主要如下所示：

•1. RSA（所有密钥大小），AES，DES / 3DES，ECC，HMAC，MD5，SHA-1，SHA-2（SHA-224，SHA-256，SHA-384，SHA-512）

•2. 带有ECDSA（签名）和ECDH（加密）的椭圆曲线密码术（ECC），包括P曲线192/224/256/384/521

- 为了更好的兼容性，本平台已经完成了对mbedtls 3.3.0安全库的适配，目前默认使用的是mbedtls。

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

06 Socket Option配置示例

归芯科技

- Socket相关参数配置：

1.Send timeout/Recv timeout/Recv windows可通过接口配置：

```
setsockopt(INT sockID, INT option_level, INT option_name, const VOID *option_value, INT option_length)
```

SO_SNDTIMEO：Send timeout option

SO_RCVTIMEO：Recv timeout option

SO_RCVBUF：Recv windows option

例如：配置send timeout为5s

```
struct timeval timeout;
timeout.tv_sec = 5;    //单位:秒
timeout.tv_usec = 0;
setsockopt(sock, SOL_SOCKET, SO_SNDTIMEO, &timeout, sizeof(timeout));
```

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

07 wireshark包抓取

归芯科技

Wireshark包抓取

1)ap：可以指定抓取蜂窝网卡、USB网卡、PPP网卡的网络包

开启：

抓取全部网卡的CAP：AT+CAPAPDATA=”mobile”,”usb”,”ppp”,<length>

抓取蜂窝网卡的CAP: AT+CAPAPDATA=”mobile”,”“,”“,<length>

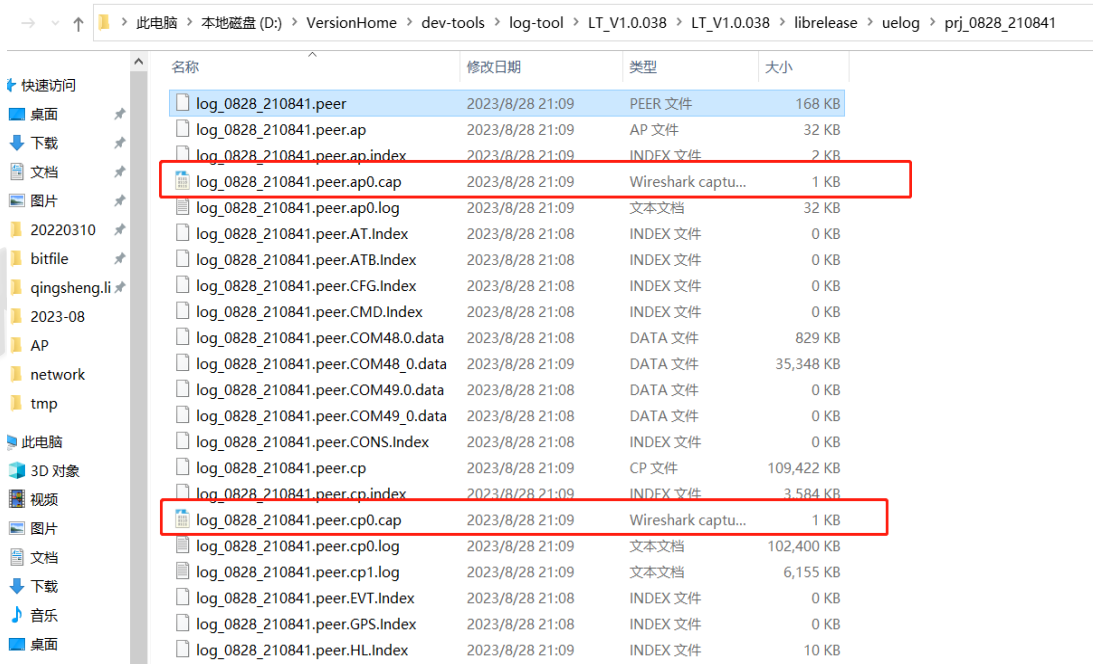
关闭：AT+CAPAPDATA=”“,”“,”“,

2)cp:

开启：AT^AEM=9,”cpcapon <length>”

关闭：AT^AEM=9,”cpcapoff”

最终cap log的文件和LOG工具抓取的其他log在同一目录。



Confidential @ 2022 Guixin Technology Co., Ltd.



归芯科技

AT

Confidential

深圳 · 上海 · 北京 · 天津 · 成都



1 AT命令接收和处理框架

2 AT框架使用介绍

3 AT命令超时处理机制

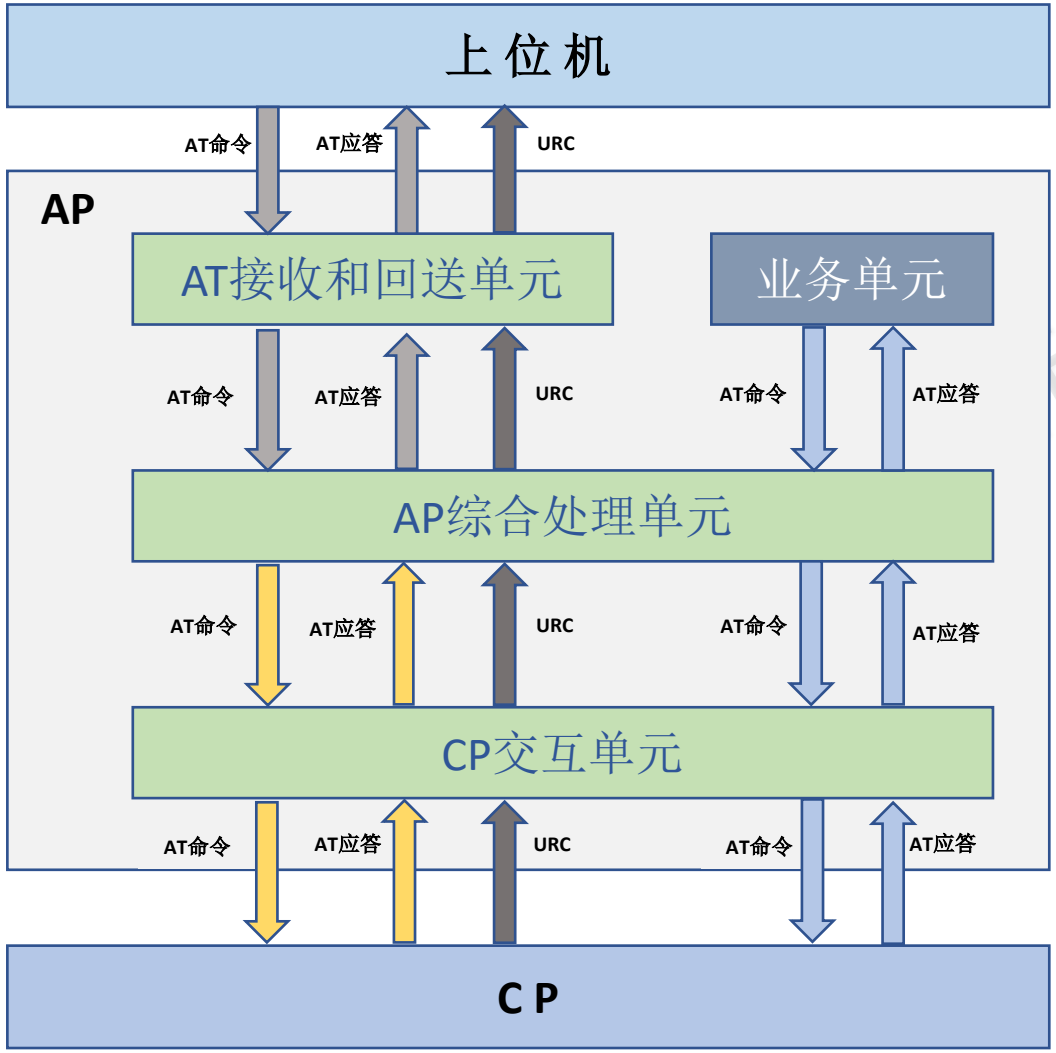
4 AT发送IP数据的补充说明

5 AT框架API使用示例

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

AT命令接收和处理框架



- 从上位机收到的AT命令，AP综合处理单元会区分其是在AP还是CP处理，需要在AP处理的会调用其处理函数，需要CP处理的则自动发给CP。
- 图中不同颜色的箭头分别对应四种使用场景。

场景一

场景二

场景三

场景四

Confidential @ 2022 Guixin Technology Co., Ltd.

AT框架使用介绍

归芯科技

➢ 场景一：从串口收到AT命令，并在AP处理该命令，步骤如下（详见aic_at_process_demo.c）：



AICXTEK Confidential

➢ 场景二：从串口收到AT命令，该命令需要发给CP，该情形框架会自动处理。

Confidential @ 2022 Guixin Technology Co., Ltd.

AT框架使用介绍

归芯科技

- 场景三：AP的某业务模块需要发送AT命令到CP，并获取和处理从CP收到的response。

1）异步接口，使用callback函数接收response，调用后即返回，不等待。

```
u32 aic_send_cmd_with_callback(u8 *at_str, aic_get_cp_resp_callback cb);
```

2）同步接口，调用后会一直等待，收到CP的response后才会返回，response保存到传入的buffer。

```
u32 aic_send_cmd_and_wait(u8 *at_str, AT_RESP_FROM_CP *resp_to_save);
```

3）获取到response和response的line count后，可以使用如下两个接口解析参数。

```
ulong *aic_get_each_line_from_resp(u8 *at_resp, u16 line_count);
ulong *aic_split_paras_to_str_arrar(u8 *raw_str, u8 *start_str, u16 *paras_cnt);
```

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

AT框架使用介绍

归芯科技

➢ 场景四：在AP侧处理CP发给AP的主动上报（URC）

1）使用add URC handle的方式，可以接收和处理特定的URC。



AICXTEK Confidential

2）使用如下接口注册URC callback，可以收到所有的URC。

u32 aic_register_cp_urc_callback(aic_get_cp_urc_callback cb);

Confidential @ 2022 Guixin Technology Co., Ltd.

AT命令超时处理机制

归芯科技

- 上位机发给模组的AT命令，超时后给上位机回复ERROR。

1）如该命令是CP处理的，则默认3s超时，对一些比较耗时的命令，其超时时间是6s、10s或30s不等。

2）如该命令是AP处理的并且有耗时的操作，则其超时时间依赖该命令自身的处理机制。

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

AT发送IP数据的补充说明

归芯科技

- 对于需要发送IP数据的AT命令，如AT+CIPSEND命令，收到该命令后，由于接下来要等待上位机发送数据，所以在其处理函数中需要调用如下接口将该命令设置为AT_DATA_MODE；待检测到上位机发送数据完毕，将该命令重置为AT_CMD_MODE。 不发送IP数据的AT命令无需使用此接口）

void aic_set_at_handle_mode (AT_REQ_RESP *at_req_resp, u8 at_mode);

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

AT框架API使用示例

归芯科技

- 关于AT框架API的使用，可以参考如下demo文件aic_at_process_demo.c。



AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.



归芯科技

Uart

Confidential

归芯科技

目录

1 Uart Hardware

2 Uart Interface

3 Uart Config

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

Uart Hardware

- Support six uart port in chip.

Description	UART Signals Matrix					
KeyWord	UART_Matrix					
Module Signals	RTS=O=1,TXD=1,RXD=1,CTS=I=1					
Device Name	UART0	UART1	UART2	UART3	UART4	UART5
line	2line	4line	2line	4line with IrDA	4line	8line with IrDA
Hardware Pin Name	UART3	UART4	UART5	UART0	UART1	UART2
IntF0				0(Default) --下载		
IntF1					1(Default) --下载	
IntF2						2(Default, Custom use)
IntF3	3(Default) --AP Log					
IntF4		4(Default)				
IntF5			5(Default)			
System	MCU			AON	TOP	
sleep	Disable, not support wakeup, and receive data is lost.	Disable, not support wakeup, and receive data is lost.	Disable, rx can wakeup mcu and the data will be lost	enable, wakeup by uart irq clock: 32K baudrate: max support 8192	Disable, not support wakeup, and receive data is lost.	Disable, rx or cts can wakeup mcu and the data will be lost.
clock	100M, 26M(Default)	100M, 26M(Default)	100M, 26M(Default)	100M, 26M(Default), 32K	100M, 26M(Default), 32K	100M, 26M(Default), 32K
Baudrate	Max 10M Baudrate	Max 10M Baudrate	Max 10M Baudrate	Max 10M Baudrate	Max 10M Baudrate	Max 10M Baudrate
Adaptive baudrate	Not Support	Not Support	Not Support	Not Support	Not Support	Not Support

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

UART0

归芯科技

- **Hardware Name：**

  UART3

- **Pin:**

  Support 2 line pin control, include txd、rxd.

- **Mode:**

  Support Normal mode.

- **Sleep:**

  Disable in deep sleep and not support wakeup.

- **Default:**

  Use for ap log.

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

UART1

归芯科技

- **Hardware Name：**

  UART4

- **Pin:**

  Support 4 line pin control, include txd、rxd、rts、cts.

- **Mode:**

  Support Normal mode, and support hardware flow control.

- **Sleep:**

  Disable in deep sleep and not support wakeup.

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

UART2

归芯科技

- **Hardware Name：**

  UART5

- **Pin:**

  Support 2 line pin control, include txd、rxd.

- **Mode:**

  Support Normal mode, and support hardware flow control.

- **Sleep:**

  Disable in deep sleep. If mcu in deep sleep, receive data only use for wakeup mcu and the data will be lost.

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

UART3

归芯科技

- **Hardware Name：**

  UART0

- **Pin:**

  Support 4 line pin control, include txd、rxd、rts、cts.

- **Mode:**

  Support Normal and Irda mode, and support hardware flow control.

- **Sleep:**

  Can work on deep sleep use 32K clock.

- **Default:**

  Use for download.

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

UART4

归芯科技

- **Pin:**

Support 4 line pin control, include txd、rxd、rts、cts.

- **Mode:**

Support Normal mode, and support hardware flow control.

- **Sleep:**

Disable in deep sleep and not support wakeup.

- **Default:**

Use for cp log and download.

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

UART5

归芯科技

- **Pin:**

Support 8 line pin control, include txd、rxd、rts、cts、dtr、dsr、dcd、ri.

- **Mode:**

Support Normal 、 Modem DTE、 Irda mode, and support hardware flow control.

- **Sleep:**

Disable in deep sleep. If mcu in deep sleep, receive data only use for wakeup mcu and the data will be lost.

- **Default:**

Use for extern mcu connect.

AIC XTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

UART Interface

归芯科技

- **Include**

  serial_inc.h

- **Interface**

  void *uart_open(int com);

  void uart_close(void *handle);

  size_t uart_read(void *handle, void *buffer, size_t size);

  size_t uart_write(void *handle, const void *buffer, size_t size);

  void uart_set_baud_rate(void *handle, int rate);

  void uart_set_fctrl(void *handle, u32 val);

AIOXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

UART Config

归芯科技

- board/config/serial/serial_config.h

User can config baudrate、cd、dma、trigger、mode、fcm and so. on.

```
..name = "uart0",
..inf = SERIAL_INF3,
..dst_inf = SERIAL_INF3,
..base = REG_BASE_AP_UARTC0,
..baud = 2000000,
..source_clk = SERIAL_SOURCE_CLK,
..irq = IRQ_UART0,
..cd = 0x1,
..wsize = SERIAL_WSIZE1,
..tdmaeb = SERIAL_DMA_ENABLE,
..thwid = DMA_REQ_MCU_UART0_TX,
..ttr = SERIAL_FIFO_T_LEVEL,
..rdmaeb = SERIAL_DMA_DISABLE,
..rhwid = DMA_REQ_MCU_UART0_RX,
..rtor = SERIAL_TIMEOUT_CYCLE,
..rtr = SERIAL_FIFO_R_LEVEL,
..IRmode = SERIAL_UART,
..uclken = SERIAL_SRC_UCLK,
..parity = NO_PARITY,
..fcm = SERIAL_FLOW_CTRL_DISABLE,
..fcdr = SERIAL_FIFO_R_FCTL,
..eb_reg = REG_MCU_GLB_UART_0_EB,
..eb_bit = BIT_MCU_GLB_UART0_EB,
..rst_reg = REG_MCU_GLB_SW_RST_GLB_CLK_MCU_UART0,
..rst_bit = BIT_MCU_GLB_SW_RST_GLB_CLK_MCU_UART0,
```

Confidential @ 2022 Guixin Technology Co., Ltd.



归芯科技

GPIO

Confidential

深圳 · 上海 · 北京 · 天津 · 成都

目录

1 GPIO Interface

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

GPIO interface

- **Include**

gpio_inc.h aicos\drivers\include

- **Interface**

```
typedef enum {
    GPIO_OUTPUT_LVL_LOW,
    GPIO_OUTPUT_LVL_HIGH,
    GPIO_TRIGGER_LVL_LOW,
    GPIO_TRIGGER_LVL_HIGH,
    GPIO_TRIGGER_EDGE_RISING,
    GPIO_TRIGGER_EDGE_FALLING,
    GPIO_TRIGGER_EDGE_ANY,
    GPIO_BYPASS,
    GPIO_INPUT_NOINT
} GPIO_MODE;

void aic_gpio_output(GPIO_PIN_ID index, GPIO_MODE mode);
u32 aic_gpio_level_get(GPIO_PIN_ID index);
void aic_gpio_input(GPIO_PIN_ID index, GPIO_MODE mode, GPIO_CBFUN gpio_cb, void *param);
int aic_gpio_request(GPIO_PIN_ID index);
void aic_gpio_free(GPIO_PIN_ID index);
```

Confidential @ 2022 Guixin Technology Co., Ltd.



归芯科技

Debug

Confidential

深圳 · 上海 · 北京 · 天津 · 成都

归芯科技

目录

1 Log

2 Memory Dump

3 T32 Simulator

4 T32 Online Debug

AICXTEK Confidential

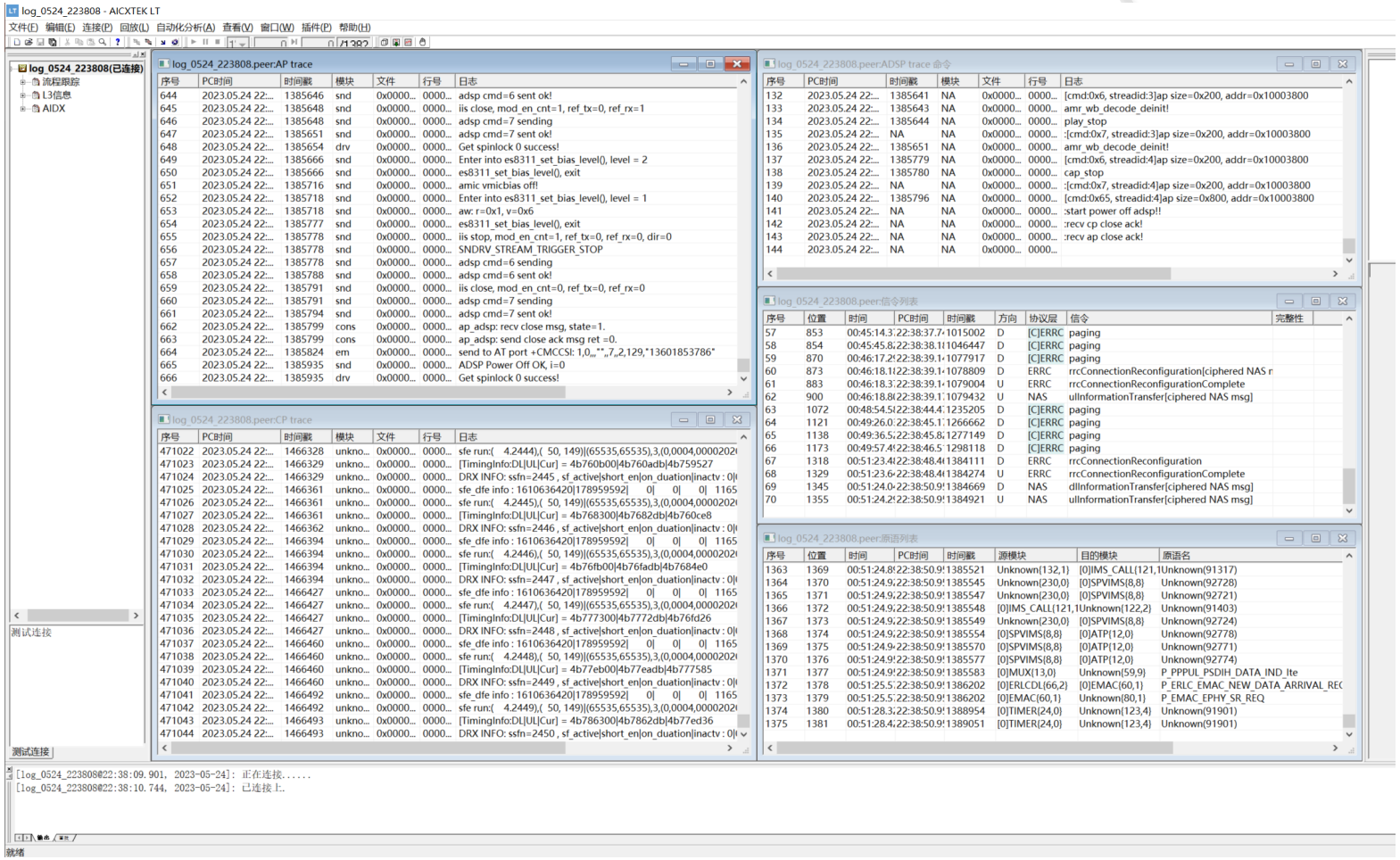
Confidential @ 2022 Guixin Technology Co., Ltd.

Debug

归芯科技

- **Log**

开机后连接LT工具，可以同时抓取AP/CP的Log（使用方法可以参考LT工具目录下的使用文档）



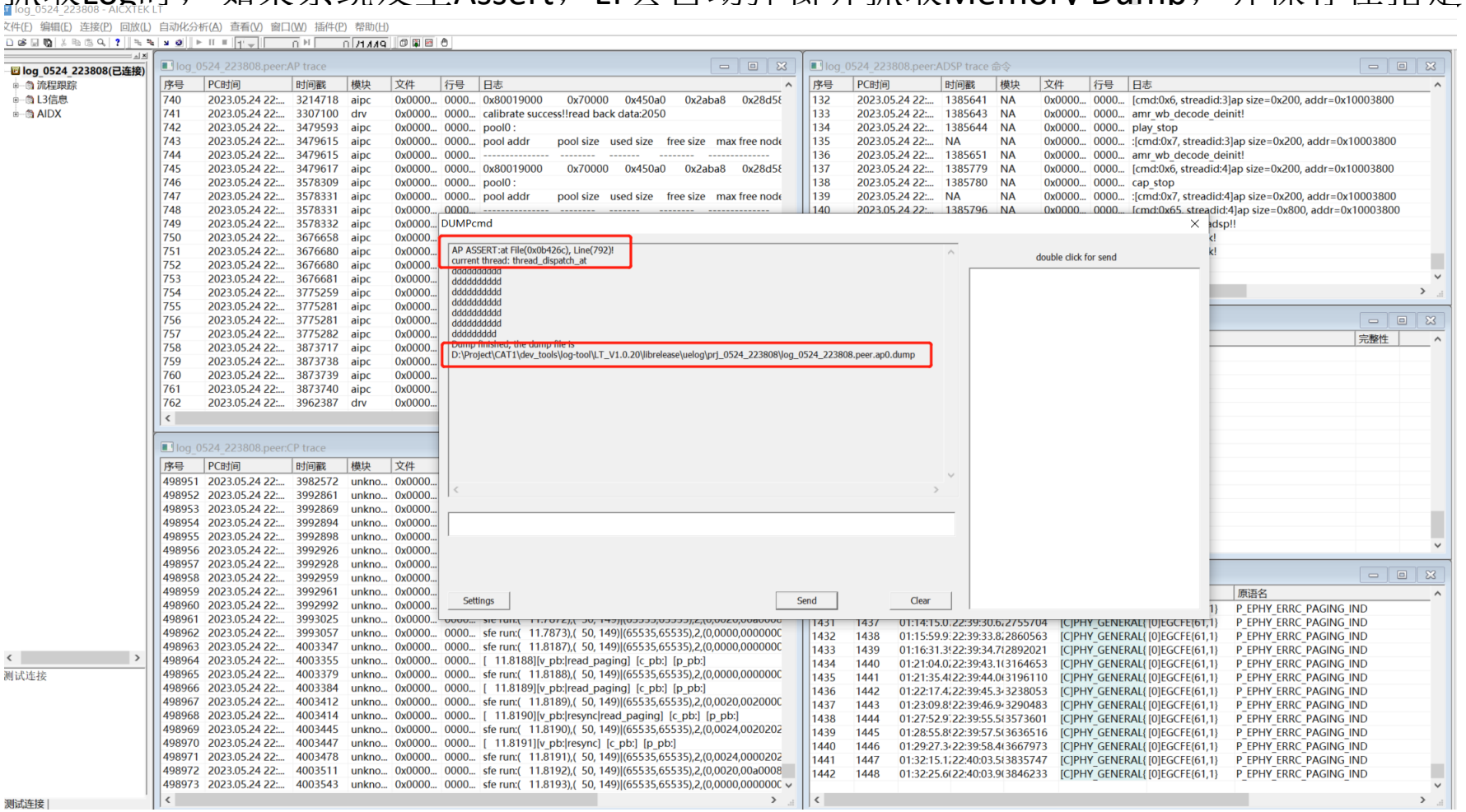
Confidential @ 2022 Guixin Technology Co., Ltd.

Debug

归芯科技

- **Memory Dump**

在使用LT工具抓取Log时，如果系统发生Assert，LT会自动弹窗并抓取Memory Dump，并保存在指定目录下。



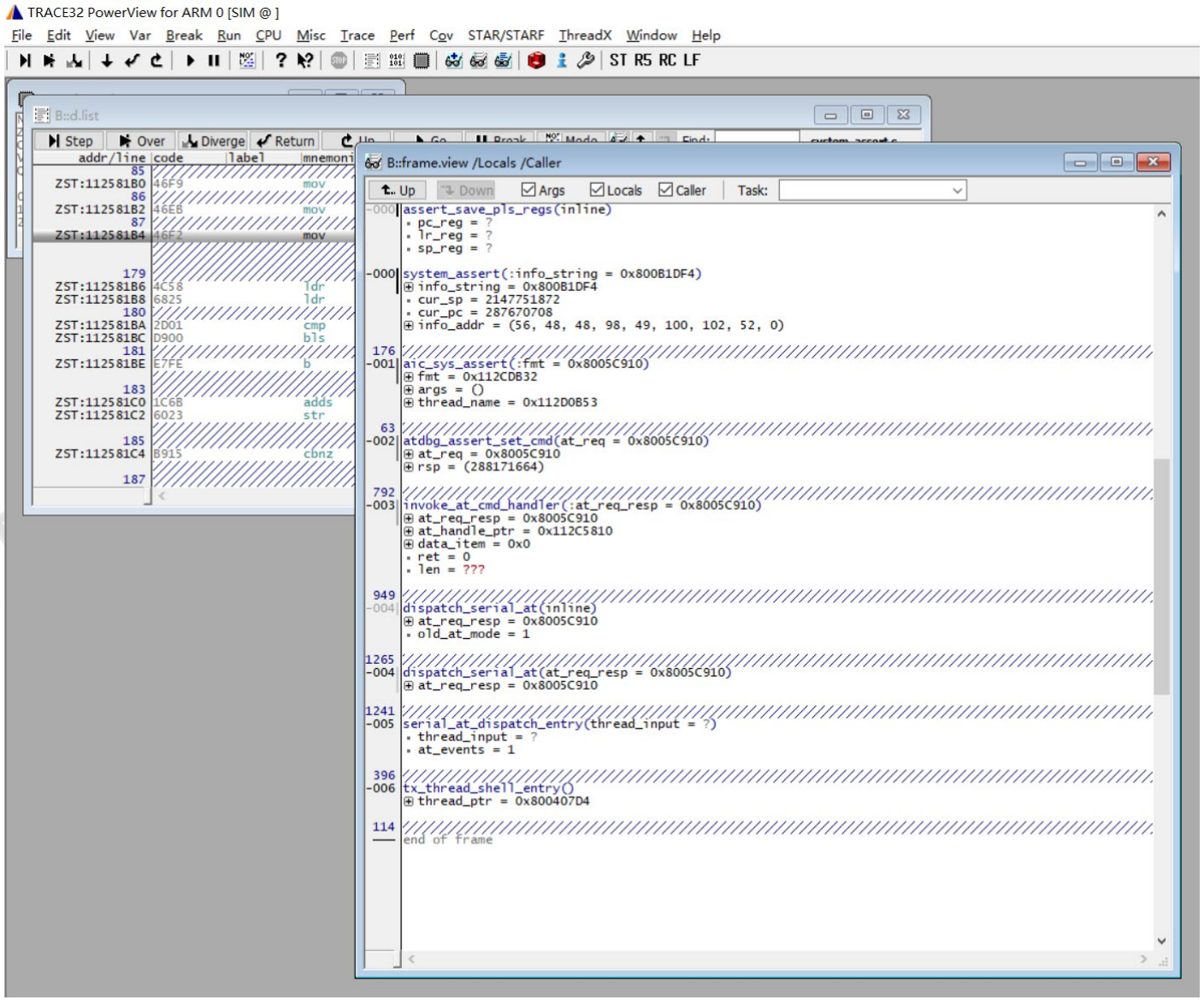
Confidential @ 2022 Guixin Technology Co., Ltd.

Debug

归芯科技

- **T32 Simulator**

使用抓取的Memory Dump，以及对应的elf，通过cmm脚本可以恢复问题现场，并进行分析。



AICXT

Confidential @ 2022 Guixin Technology Co., Ltd.

Debug

归芯科技

- **T32 Online Debug**

对于USB还未枚举之前的死机问题，无法通过LT工具进行抓取，可以用T32连接板子的JTAG口进行在线分析。

1. 通过CMM脚本进行Attach;
2. 加载对应的elf；
3. 结合callstack, 全局变量等进行问题分析，也可以通过断点、单步调试等方式进行debug。

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

Camera

Camera目前支持自适应不同的sensor，对于不同的sensor工程配置文件为sensor_xxx_cfg.h，主要配置如下信息：

- Sensor输出线模式，目前支持1线模式和2线模式
- Sensor输出数据格式，包括MTK格式和展讯格式
- Sensor采样模式，包括SDR和DDR采样
- irq和PWDN/RST GPIO等
- Camera sensor电压信息，要注意电压的USER_NAME要唯一

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

Camera配置camera_cfg.h

这里以aicos/projects/genesis/ap/gx316h_h1/config/camera/camera_cfg.h为例：

```
……
//#define CAMERA_SENSOR_OUTPUT_640x480

/* sensor output data format */
//#define CAMERA_SENSOR_OUTPUT_Y(注意这个打开默认就是配置输出Y only,默认为YUV)
……
```

AICXTEK Confidential

Camera配置sensor_xxx_cfg.h

归芯科技

这里以aicos/projects/genesis/ap/gx316h_h1/config/camera/sensor_bf3901ucs_cfg.h为例：

```
/* line number define */
#define CAMERA_OUTPUT_1LINE
//#define CAMERA_OUTPUT_2LINE
//#define CAMERA_OUTPUT_4LINE

/* output data format define */
//#define CAMERA_OUTPUT_MTK
#define CAMERA_OUTPUT_Z

/* data sample mode define */
//#define CAMERA_DDR_MODE
#define CAMERA_SDR_MODE

#define CAMERA_SENSOR_FPS_90

#if defined(CONFIG_PMIC)
/* camera sensor voltage supply */
#define CAMERA_SENSOR_IO_USER_NAME    "cam_bf3901ucs_1v8"
#define CAMERA_SENSOR_IO_UNIT_NAME    "VDDCAMIO1V8"
#define CAMERA_SENSOR_AVDD_USER_NAME  "cam_bf3901ucs_2v8"
#define CAMERA_SENSOR_AVDD_UNIT_NAME  "AVDDCAM2V8"
#endif

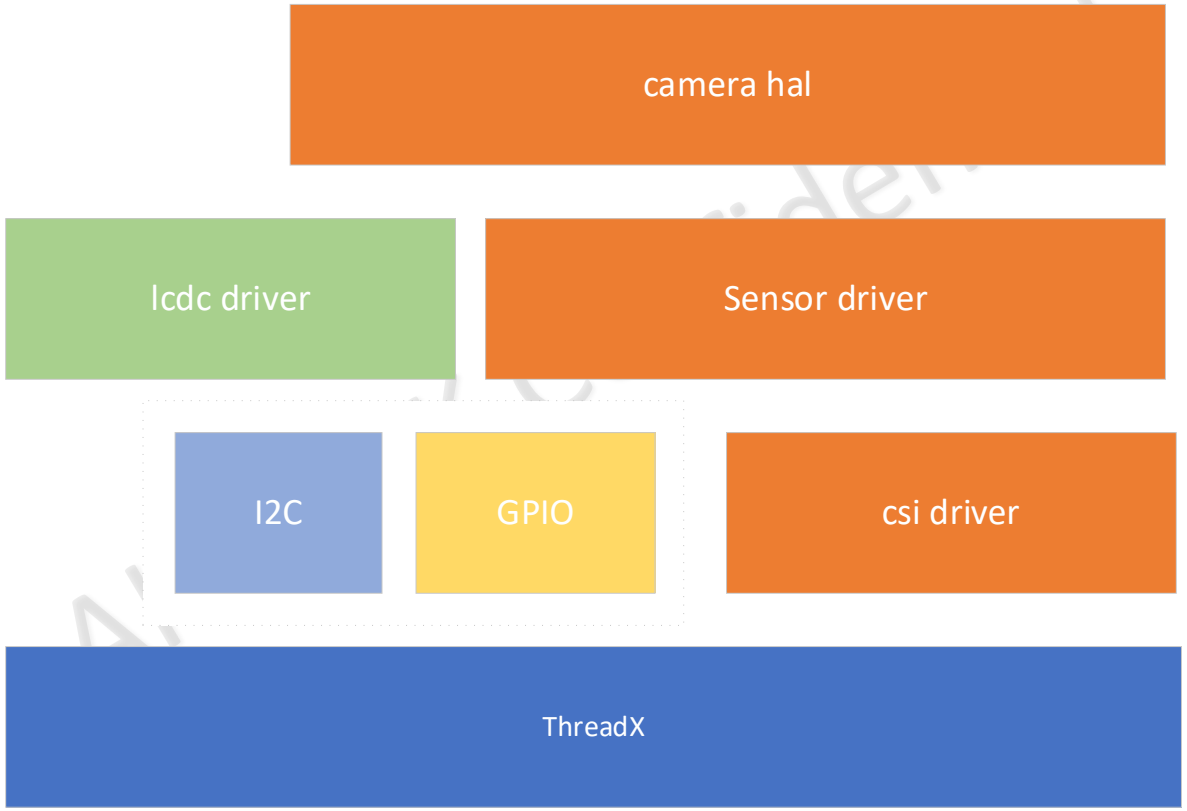
struct aic_camera_cfg camera_sensor_bf3901ucs_cfg = {
    .irq = IRQ_CSI,
    .gpio_pwdn = 2,
    .gpio_rst = 2,
    .cs_sensor_ops = AIC_NULL,
};
```

ARCYTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

Camera

归芯科技



Confidential @ 2022 Guixin Technology Co., Ltd.

Camera

归芯科技

函数原型	参数
int camera_register(preview_refresh_cb refresh)	refresh: preview callback函数
int camera_open(camera_id_e id)	id: sensor id信息
void camera_close(camera_id_e id)	id: sensor id信息
int camera_preview(camera_id_e id, yuv_plane_num_e plane_num, camera_data_fmt_e fmt, int width, int height)	id: sensor id信息 plane_num: yuv格式的plane数目 fmt: image格式 width: 图片宽度 height： 图片高度
int camera_capture_prepare(camera_id_e id, yuv_plane_num_e plane_num, camera_data_fmt_e fmt, int width, int height, u32 user_addr)	id: sensor id信息 plane_num: yuv格式的plane数目 fmt: image格式 width: 图片宽度 height： 图片高度 user_addr: 用户提供的地址信息，抓取的图片会拷贝到该位置
int camera_capture(camera_id_e id)	id: sensor id信息
int camera_ioctl(camera_id_e id, camera_cmd_e cmd, void *input, void *output)	id: sensor id信息 cmd: 可操作的指令 input： 指令参数信息 output： 指令输出信息

Confidential @ 2022 Guixin Technology Co., Ltd.

Camera

```
typedef enum {
    CAMERA_FRONT = 0,
    CAMERA_BACK,
} camera_id_e;
```

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

Camera

归芯科技

```
typedef enum {
    YUV_ONE_PLANE = 0,
    YUV_TWO_PLANE,
} yuv_plane_num_e;
```

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

Camera

归芯科技

```
typedef enum {
    YUV422 = 0,
    YUV420,
    RAW8,
    RGB565,
    Y,
} camera_data_fmt_e;
```

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

Camera

归芯科技

```
typedef enum {
    CAMERA_STREAM_ON = 0,
    CAMERA_STREAM_OFF,
    ……
} camera_cmd_e;
```

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

Camera

归芯科技

……

```
/* 注册preview callback */
int camera_register(preview_refresh_cb refresh);
/* open后会默认根据lcd size打开预览模式 */
camera_open(CAMERA_FRONT);
/* 如果有特殊需求，可以根据需求配置preview信息 */
int camera_preview(camera_id_e id, yuv_plane_num_e plane_num,
                   camera_data_fmt_e data_fmt,
                   int width, int height);
/* 配置capture信息 */
camera_capture_prepare(CAMERA_FRONT, YUV_ONE_PLANE, YUV, width, height, 0x80100000);
/* 抓取一帧图片保存到用户地址 */
camera_capture(CAMERA_FRONT);
camera_close(CAMERA_FRONT);
```

AICXTEK Confidential

Confidential @ 2022 Guixin Technology Co., Ltd.

归芯科技

谢 谢

臻于互联 万物归芯



ACXTEK Confide

深圳 · 上海 · 北京 · 天津 · 成都